



POLITECNICO
MILANO 1863

NUMERICAL OPTIMIZATION FOR CONTROL

Distributed-MPC Control for Multi-UAV Map Coverage

Authors:

Francesco STREET	10769571
Andrea VENEZIA	10797678
Francesco Urbano SERENO	10849720

Contents

1	Abstract	2
2	Introduction	2
3	System Modeling	4
4	Problem Formulation	5
4.1	Environment Representation	5
4.2	Waypoint Generation and Sanitization	6
4.3	Task allocation: Voronoi Tessellation	7
4.4	Trajectory Sequencing (TSP and 2-Opt)	9
5	Model Predictive Control Strategy	10
5.1	Cost Function Formulation	11
5.2	Waypoint Tracking	12
5.3	Smoothness and Control Effort (Jerk)	12
5.4	Energy Efficiency and Altitude Maintenance	12
5.5	Obstacle Avoidance and Safety Slacks	13
5.5.1	Linearization via Separating Hyperplanes	13
6	Results	14
7	Improvements	18
8	Robustness Analysis	21
9	Conclusions	24

1 Abstract

This report presents a distributed optimization framework for multi-UAV coverage of three-dimensional environments. The coverage task is decomposed into waypoint allocation and trajectory generation, where each agent solves a local Quadratic Program (QP) within a Distributed Model Predictive Control (DMPC) scheme.

2 Introduction

Unmanned Aerial Vehicles (UAVs) offer significant operational advantages, such as small size, high maneuverability, and low cost, making them highly suitable for coverage and search missions. However, the efficacy of a single UAV is frequently hindered by strict constraints regarding battery endurance and communication range of the remote control. Consequently, deploying a coordinated multi-agent system—or swarm—of UAVs emerges as a robust solution to overcome these limitations.

In the scope of this project are considered UAVs with H.2 hovering capabilities (e.g. quadcopters), not fixed wing ones.

The primary objective of this project is to coordinate a fleet of N_u UAVs to execute coverage and inspection tasks within a three-dimensional (3D) environment populated by static cylindrical obstacles. The mission requires the swarm to maximize the coverage of a known map while minimizing the energy consumption and control effort.

Since the map is known, it is assumed that the radius of the cylindrical approximation of the obstacles is well known for all the agents. This will prove crucial in the obstacle avoidance constraints in the optimization problem.

Crucially, the control strategy must ensure strict safety guarantees throughout the mission by continuously preventing both inter-agent collisions and collisions with environmental obstacles. To perform the inspection task, each UAV is equipped with a camera characterized by a known Field of View (FOV) and aspect ratio, which defines its sensory footprint within the environment.

In the following, it is assumed that the local UAV controller is already implemented and takes care of the input allocation to apply the optimal control action. Furthermore, the UAVs are assumed to be equipped with an IMU, together with GNSS tracking. Such sensors and measures are integrated with

an appropriate Sensor Fusion technique, allowing position estimation to be 100% accurate initially, and then subject to a variable noise between 20-30 centimeters in the robustness analysis.

Finally, to perform successfully the DMPC algorithm, communication between agents needs to be fully operational within the boundaries of the map, or at least in a certain range to collect and transmit part of the messages of interest to other agents in a multiagent system fashion.

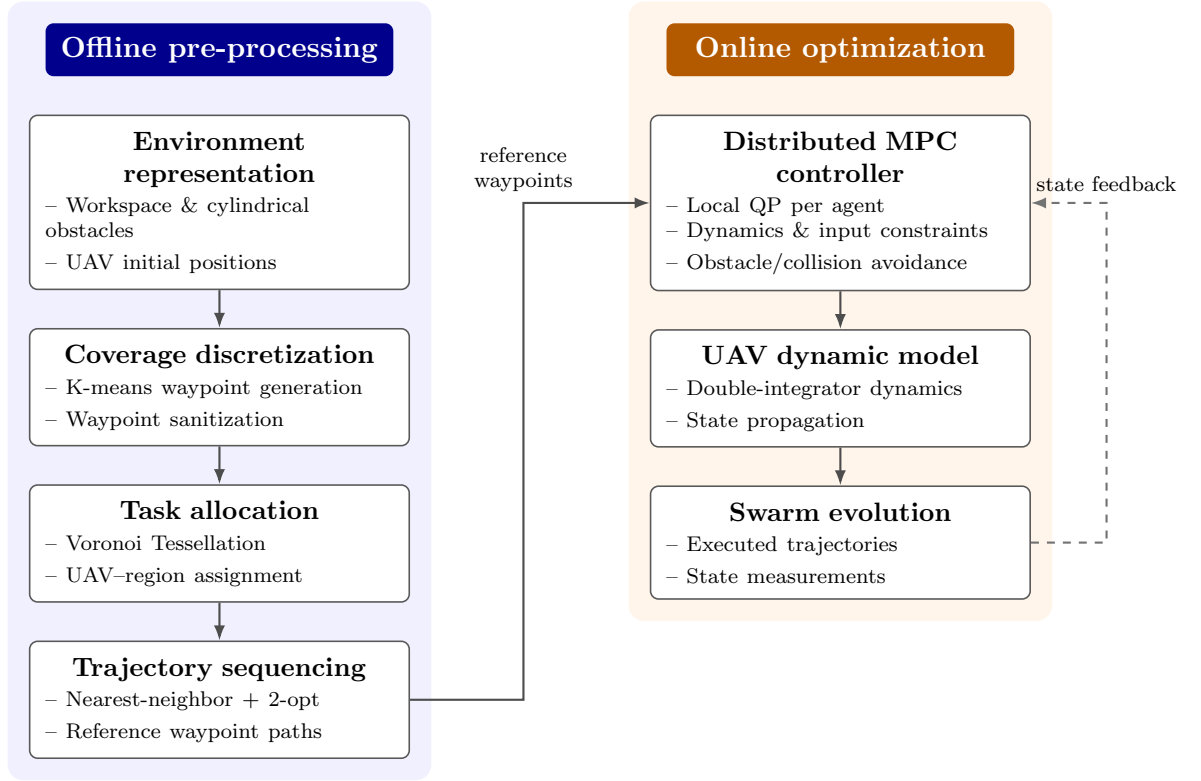


Figure 1: Overview of the proposed framework.

3 System Modeling

The trajectories generation problem is formulated via Model Predictive Control (MPC), which requires a discrete-time representation of the UAV dynamics. Each UAV is modeled as a point-mass system with double-integrator dynamics in a 3D environment, providing a linear approximation of the vehicle motion while maintaining a suitable trade-off between model accuracy and computational efficiency.

The state vector of each UAV at time instant k is defined as

$$\mathbf{x}_k = [\mathbf{p}_k^T \quad \mathbf{v}_k^T]^T \in R^6, \quad (1)$$

where $\mathbf{p}_k = [x_k, y_k, z_k]^T \in R^3$ denotes the position of the UAV and $\mathbf{v}_k = [v_{x,k}, v_{y,k}, v_{z,k}]^T \in R^3$ denotes its velocity.

The control variable is chosen as the acceleration input

$$\mathbf{u}_k = \mathbf{a}_k \in R^3. \quad (2)$$

Assuming that the acceleration remains constant during the sampling interval Δt , in order to correctly perform the OSQP-based optimization, the continuous-time dynamics are discretized using a zero-order hold approximation. The resulting discrete-time model is

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k \Delta t + \frac{1}{2} \mathbf{a}_k \Delta t^2 \quad (3)$$

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \mathbf{a}_k \Delta t \quad (4)$$

Finally, physical limitations of the UAV are included in the optimization problem through bounds on the velocity and acceleration. These constraints are expressed as

$$\|\mathbf{a}\| \leq a_{max} \quad \|\mathbf{v}\| \leq v_{max} \quad (5)$$

where $v_{\max}$ and $a_{\max}$ represent the maximum admissible velocity and acceleration, respectively.

4 Problem Formulation

The initial coverage task is formulated over a three-dimensional environment, with the objective of generating collision-free trajectories maximizing the observed region by the swarm while satisfying constraints.

The direct optimization of the coverage during the online control phase would require evaluating the projected camera footprint on the workspace captured by each UAV's camera, leading to an extreme computational effort.

To circumvent the computational bottleneck, the coverage problem is decomposed into two main stages. An offline preprocessing phase is introduced to transform the continuous coverage objective into a finite waypoint-reaching problem. The obstacle-free space within the map is partitioned into a set of discrete sub-regions, each possessing an area approximately equivalent to the area covered by a single camera. Heuristically, this reduces the complex continuous coverage problem to a discrete waypoint-reaching task: if the UAV swarm successfully visits the centroid of every sub-region, the global coverage objective is considered fulfilled.

The resulting waypoint sequences are then used as references for the online Model Predictive Control (MPC) optimization, which computes dynamically feasible trajectories while enforcing safety constraints.

4.1 Environment Representation

The mission environment is represented as a bounded three-dimensional workspace containing static cylindrical obstacles. Cylindrical obstacles are characterized by a convex geometry, which simplify collision checking, constraint formulation, and the generation of diverse scenarios while keeping the optimization problem numerically tractable.

The workspace is defined as:

$$\mathcal{E} = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \times [z_{\min}, z_{\max}], \quad (6)$$

where the three intervals define the admissible flight region along the Cartesian axes. The obstacle set is composed of N_o cylindrical objects:

$$\mathcal{O} = \{O_1, O_2, \dots, O_{N_o}\}. \quad (7)$$

Each obstacle O_j is characterized by its center position (x_j, y_j) , radius r_j , and height h_j . For the preprocessing stage, the cylindrical obstacles are projected onto the horizontal plane, obtaining the circular footprint

$$O_j^{xy} = (x, y) \in R^2 : \|(x, y) - (x_j, y_j)\|_2 \leq r_j. \quad (8)$$

This projection allows the coverage problem to be reformulated in a two-dimensional workspace, reducing the computational complexity of waypoint generation and task allocation.

The corresponding free space is therefore defined as

$$\mathcal{F}_{xy} = \mathcal{E}_{xy} \setminus \bigcup_{j=1}^{N_o} O_j^{xy}. \quad (9)$$

The obstacle height is still considered during the trajectory optimization phase, where the collision avoidance constraints take into account the z-axis motion of the UAVs.

The parametric generation of the environment enables the creation of multiple scenarios by varying workspace dimensions, obstacle density, and obstacle geometry. This feature allows a systematic evaluation of the proposed control strategy under different levels of complexity and supports robustness analysis over randomly generated environments.

4.2 Waypoint Generation and Sanitization

In order to guarantee computational feasibility and to avoid an increasing complexity of the computational problem, the continuous coverage problem is reformulated as a finite waypoint-reaching problem during the preprocessing stage. The camera field of view is projected onto the horizontal plane assuming a reference flight altitude, obtaining the corresponding sensing footprint area A_{FOV} .

The required number of waypoints is then estimated as

$$N_w = \left\lceil \frac{A_{\text{map}}}{A_{\text{FOV}}(1 - \gamma)} \right\rceil, \quad (10)$$

where A_{map} is the horizontal area of the environment, and γ is an overlap factor introduced to improve coverage robustness. Increasing γ enlarges the

overlap between adjacent sensing regions, that reduces the uncovered areas, but increases the number of waypoints leading to a higher computational cost in the later stages. So, the tuning of this parameter is achieved comparing the performances of the control algorithm in terms of coverage and energy consumption.

From the continuous free-space is generated a finite set of representative waypoints by applying the **K-means clustering** algorithm. The clustering centroids are used as target locations:

$$\mathcal{W} = \mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{N_w}, \quad (11)$$

where $\mathbf{w}_i \in R^2$ represents the i -th *waypoint*. K-means is preferred respect to other approaches, such as hierarchical clustering or density-based methods, due to its low computational complexity and direct capability of tuning the desired number of waypoints without adding further tuning parameters.

However, to guarantee the feasibility of the trajectories generated by the MPC and to prevent numerical conflicts, a **sanitization** procedure is applied to take into account obstacle clearance. Each waypoint is checked against the obstacle set, and points located inside the safety region of an obstacle are pushed radially until the minimum admissible distance is satisfied:

$$\|\mathbf{w}_i - \mathbf{o}_j\|_2 \geq r_j + d_{\text{safe}}, \quad (12)$$

where $\mathbf{o}_j$ and r_j represent the *center* and *radius* of the j -th obstacle, and d_{safe} is the *safety margin*.

4.3 Task allocation: Voronoi Tessellation

After generating the waypoints set $\mathcal{W}$, targets need to be assigned among the UAVs. To balance the workload across the swarm, so to limit the overlap of already seen areas, a spatial partitioning approach has been selected. In particular, **Voronoi Tessellation** has been adopted in order to obtain finite sets of way-points that minimizes the overlaps and the idle times of each UAV.

The workspace is partitioned into N_u regions, where N_u is the *number of UAVs*. First, **K-means clustering** is applied to the set of waypoints with

a number of clusters K equal to the swarm size:

$$K = N_u. \quad (13)$$

The resulting centroids are used as generators of the Voronoi algorithm.

$$\mathcal{S} = \mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_{N_u}, \quad (14)$$

where $\mathbf{s}_i \in R^2$ denotes the seed point associated with the i -th partition.

The Voronoi partition divides the workspace into cells according to the closest generator. Each waypoint is then assigned to the Voronoi cell containing its position, obtaining a subset of targets for each UAV. This procedure also helps avoiding inter-agents collisions, dividing the map in drone-specific areas.

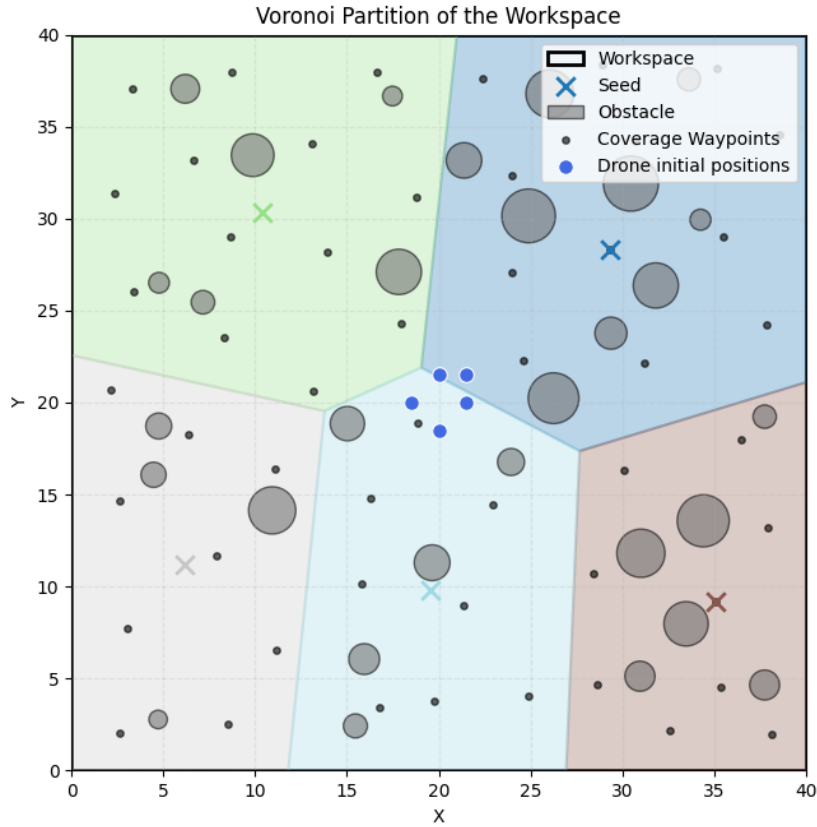


Figure 2: Example of a preprocessed map of 40x40 meters with 30 obstacles ($K = 51$ in this case)

Each partition is then assigned to the respective agent minimizing the initial travel distance of each agent, since Voronoi generators are not always aligned with the initial position of each agent. This step reduces unnecessary repositioning at the beginning of the task.

4.4 Trajectory Sequencing (TSP and 2-Opt)

After associating to each UAV the subset of way-points to be visited, the visiting order is optimized.

This problem is formulated as a **Traveling Salesman Problem (TSP)**, where the cost is the distance traveled between consecutive way-points.

$$\min \sum_{i=0}^{n-1} \|\mathbf{p}_{wp,i+1} - \mathbf{p}_{wp,i}\| \quad (15)$$

Since solving the TSP to optimality is computationally expensive, especially as the number of targets increases, a heuristic approach is adopted.

An initial route is built with the **Nearest Neighbor** heuristic, starting from the UAV initial position and repeatedly selecting the closest waypoint that has not yet been visited. This first solution is then improved with a **2-opt** local search, which reduces the total travel distance by swapping path segments and removing unnecessary crossings.

The obtained sequences are used as references to the plan of each agent. Later, during the online phase, the controller is responsible of generating feasible and smooth trajectories, while satisfying dynamical constraints and collision avoidance constraints.

5 Model Predictive Control Strategy

The navigation and collision avoidance logic is implemented through a **Distributed Model Predictive Control (DMPC)** scheme. At each control cycle, every UAV solves a local **Quadratic Programming (QP)** optimization problem over a finite prediction horizon $N = 50$ steps. The solver computes an optimal sequence of control inputs (accelerations) $\mathbf{a}_{0 \dots N-1}$, but only the first command is applied before the horizon shifts forward in a receding-horizon fashion. The chosen timestep length dt was chosen to be 50 milliseconds.

At each time step, each UAV solves the following constrained QP problem. Letting $\mathbf{p}_k, \mathbf{v}_k, \mathbf{a}_k \in R^3$ denote the position, velocity, and acceleration at predicted time step k :

$$\min_{\substack{\mathbf{p}, \mathbf{v}, \mathbf{a}, \\ \epsilon^{\text{obs}}, \epsilon^{\text{neigh}}}} w_p J_{\text{wp}} + w_{\text{eff}} J_{\text{effort}} + w_z J_z + w_{\text{batt}} J_{\text{batt}} + w_{\text{slack}} J_{\text{slack}} \quad (16a)$$

$$\text{s.t. } \mathbf{p}_0 = \mathbf{p}_{\text{init}}, \quad \mathbf{v}_0 = \mathbf{v}_{\text{init}} \quad (16b)$$

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k dt + \frac{1}{2} \mathbf{a}_k dt^2, \quad \forall k \in \{0, \dots, N-1\} \quad (16c)$$

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \mathbf{a}_k dt, \quad \forall k \in \{0, \dots, N-1\} \quad (16d)$$

$$-\mathbf{a}_{\text{max}} \leq \mathbf{a}_k \leq \mathbf{a}_{\text{max}}, \quad \forall k \in \{0, \dots, N-1\} \quad (16e)$$

$$-\mathbf{v}_{\text{max}} \leq \mathbf{v}_k \leq \mathbf{v}_{\text{max}}, \quad \forall k \in \{0, \dots, N-1\} \quad (16f)$$

$$\mathbf{p}_{\text{min}} \leq \mathbf{p}_k \leq \mathbf{p}_{\text{max}}, \quad \forall k \in \{1, \dots, N\} \quad (16g)$$

$$\epsilon_{j,k}^{\text{obs}} \geq 0, \quad \epsilon_{i,k}^{\text{neigh}} \geq 0 \quad (16h)$$

$$\|\Delta \mathbf{p}_{j,k}^{\text{obs}}\|^2 + 2 \left(\Delta \mathbf{p}_{j,k}^{\text{obs}} \right)^T (\mathbf{p}_k - \bar{\mathbf{p}}_k) + \epsilon_{j,k}^{\text{obs}} \geq (r_{\text{safe}} + r_{j,k}^{\text{obs}})^2, \quad \forall j \in \mathcal{O} \quad (16i)$$

$$\|\Delta \mathbf{p}_{i,k}^{\text{neigh}}\|^2 + 2 \left(\Delta \mathbf{p}_{i,k}^{\text{neigh}} \right)^T (\mathbf{p}_k - \bar{\mathbf{p}}_k) + \epsilon_{i,k}^{\text{neigh}} \geq r_{\text{safe}}^2, \quad \forall i \in \mathcal{N} \quad (16j)$$

Where:

- $\bar{\mathbf{p}}_k$ is the predicted position of the ego-drone at step k from the previous MPC iteration.
- $\Delta \mathbf{p}_{j,k}^{obs} = \bar{\mathbf{p}}_k - \mathbf{p}_{j,k}^{obs}$ is the distance vector between the previous ego trajectory and obstacle j .
- $\Delta \mathbf{p}_{i,k}^{neigh} = \bar{\mathbf{p}}_k - \mathbf{p}_{i,k}^{neigh}$ is the distance vector between the previous ego trajectory and neighbor i .
- r_{safe} is the predefined safety radius of the ego drone.
- $r_{j,k}^{obs}$ is the dynamic radius of obstacle j at time step k .
- $\epsilon_{j,k}^{obs}$ and $\epsilon_{i,k}^{neigh}$ are the slack variables of the obstacle avoidance and inter-agent avoidance constraints, respectively.
- Note: $\mathbf{p}_{min}$ enforces a mathematical floor constraint of -0.1m for the Z -axis to ensure feasible initialization implementation-wise.

Both a **Quadratic Program (QP)** formulation and a **Nonlinear Program (NLP)** formulation have been tried and compared. Not linearizing the obstacle avoidance constraints didn't have any noticeable effect on the quality of the found solution, so the former was chosen also because of its convex nature (translating in a orders of magnitude faster solving time). For this reason, it is discussed and analyzed only the cast QP.

5.1 Cost Function Formulation

To achieve the mission objectives, we define a multi-objective cost function J that the solver minimizes. The total cost is a weighted sum of several components, each targeting a specific behavior:

- $w_{wp}J_{wp}$ is the waypoint tracking term;
- $w_{eff}J_{effort}$ is the control effort optimization term;
- $w_{batt}J_{batt}$ is the battery-saving term;
- w_zJ_z is the altitude maintenance term;
- $w_{slack}J_{slack}$ is the obstacle avoidance term.

Each of these components is broken down in the next few subsections.

5.2 Waypoint Tracking

The primary task is to minimize the distance to the assigned waypoints. To maintain compatibility with the QP solver, we utilize the squared Euclidean distance. Furthermore, a "target focus" strategy is implemented to ensure the drone prioritizes the immediate next waypoint in the TSP-ordered sequence:

$$J_{wp} = \sum_{k=1}^N \sum_{i=0}^{N_{wp}} (1 - f_i) \cdot \|\mathbf{p}_k - \mathbf{p}_{wp,i}\|^2 \cdot \gamma_i \quad (17)$$

Where f_i is a binary flag indicating if a waypoint has been visited, and γ_i is a focus weight vector $\in R^{N_{wp}}$. His entries are set to 0 except for $i = 0$. This term initially was considering also future waypoints to smooth out the trajectory; however, this did not have the intended effect, resulting in a "brake" for the UAVs that eventually stopped them not long after the start of their mission. The final proposed formulation takes into account trajectory's smoothness with a separate cost function term.

5.3 Smoothness and Control Effort (Jerk)

To ensure flyable, smooth trajectories are generated and reduce mechanical stress, we penalize the Jerk, defined as the variation in acceleration. This encourages smooth transitions and prevents abrupt maneuvers:

$$J_{eff} = \sum_{k=0}^{N-1} \|\mathbf{a}_k - \mathbf{a}_{k-1}\|^2 \quad (18)$$

For the first step ($k = 0$), $\mathbf{a}_{k-1}$ corresponds to the acceleration applied from the solution of the optimization problem at the previous timestep.

5.4 Energy Efficiency and Altitude Maintenance

A velocity penalty is introduced to promote energy efficiency and stable flight dynamics. While the waypoint tracking term drives the drone toward its destination, this penalty counteracts excessive kinetic energy consumption:

$$J_{batt} = \sum_{k=0}^N \|\mathbf{v}_k\|^2 \quad (19)$$

This formulation is also sensible from a logical point of view: as the distance to the target decreases, the velocity penalty naturally causes the drone to decelerate and transition into a stable hover. This behavior is highly beneficial for the inspection profile, as reducing speed at the points of interest ensures higher-quality sensory data acquisition and better map coverage, in principle.

Additionally, a dedicated term ensures the drone maintains its reference altitude $z_{ref} = 5.0$:

$$J_z = \sum_{k=0}^N (z_k - z_{ref})^2 \quad (20)$$

For the scope of this project the reference height was kept constant, but in principle could vary online if operational conditions need it. Also, the DMPC formulation is general enough so that, with appropriate modifications to the waypoint generation, setting this value to ground level could allow the use of such algorithm also in presence of mobile robots as agents instead of flying UAVs.

5.5 Obstacle Avoidance and Safety Slacks

To maintain the convexity required by the QP solver while ensuring robust safety, obstacle avoidance is implemented using a combination of linearized constraints and soft-penalty slack variables. Unlike standard rigid constraints, this formulation incorporates *Dynamic Radii*, where the total safety distance d_{total} is calculated by combining the agent’s safety buffer d_{safe} with the specific radius of the closest obstacle r_{obs} at every timestep:

$$d_{total} = d_{safe} + r_{obs} \quad (21)$$

5.5.1 Linearization via Separating Hyperplanes

Since the Euclidean distance constraint is non-convex, we apply a first-order Taylor expansion around the drone’s previous predicted trajectory $\mathbf{p}_{prev}$. This creates a linear separating hyperplane that defines the feasible half-space for each obstacle j at horizon step k :

$$\|\mathbf{p}_{prev} - \mathbf{p}_{obs}\|^2 + 2(\mathbf{p}_{prev} - \mathbf{p}_{obs})^T(\mathbf{p}_k - \mathbf{p}_{prev}) + \epsilon_{j,k} \geq d_{total}^2 \quad (22)$$

The term $\epsilon_{j,k} \geq 0$ is a quadratic L2 slack variable that prevents the solver from becoming infeasible if the drone is momentarily forced into a restricted

zone. The cost function heavily penalizes these slacks using the weight w_{slack} to ensure they are only utilized as a last resort:

$$J_{slack} = \sum_{k=1}^N \sum_{j=1}^{k_{obs}} w_{slack} \cdot \epsilon_{j,k}^2 \quad (23)$$

6 Results

These initial results are obtained assuming ideal state estimation, with no position noise applied to the agents. Under these conditions, the computed trajectories exhibit high smoothness, demonstrating the UAVs' capability to successfully navigate heavily cluttered environments.

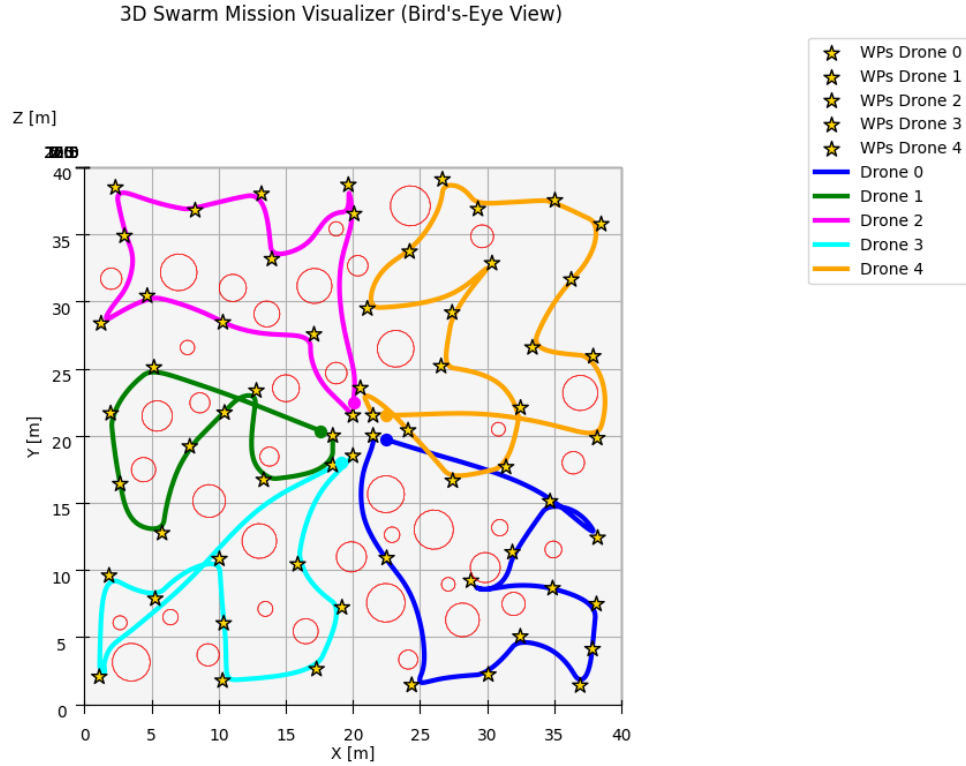


Figure 3: Computed trajectories.

To evaluate mapping performance, the sensor coverage area is modeled as a moving rectangular footprint matching the camera's Field of View (FOV). This footprint translates along the trajectory at the instantaneous velocity (magnitude and direction) of the respective drone. A representative coverage map, achieving 99.26% total coverage, is illustrated in the figure below:

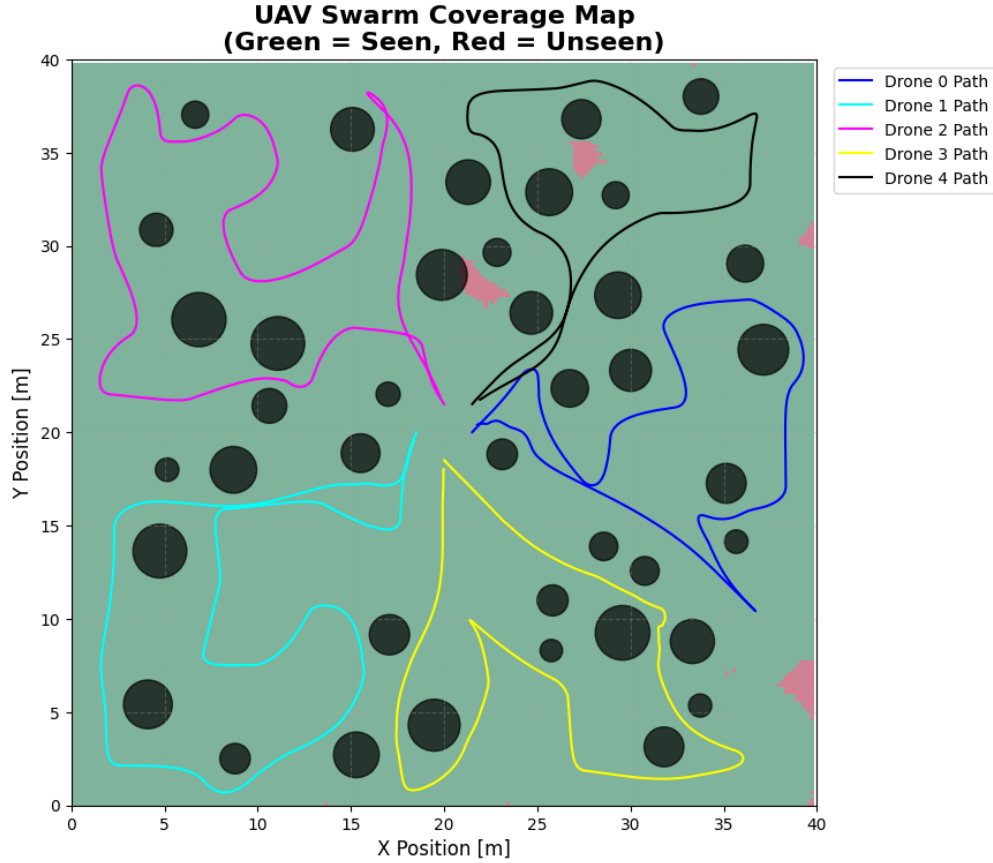


Figure 4: Coverage results - 99.26% in this case.

As anticipated, the waypoint generation procedure occasionally struggles in highly cluttered regions, resulting in minor unmapped sections, due to its heuristic nature. Nevertheless, the generated waypoints provide an effective baseline for the global coverage task, confirmed by the great results obtained.

Despite the geometric smoothness of the trajectories, the resulting con-

trol inputs exhibited high jerk. This oscillatory actuation primarily stems from three factors: the velocity penalty enforced near waypoints, the MPC’s finite prediction horizon (resulting in momentary ”blindness” to subsequent waypoints), and the aggressive cost function gradients encountered when a new target waypoint is actively assigned.

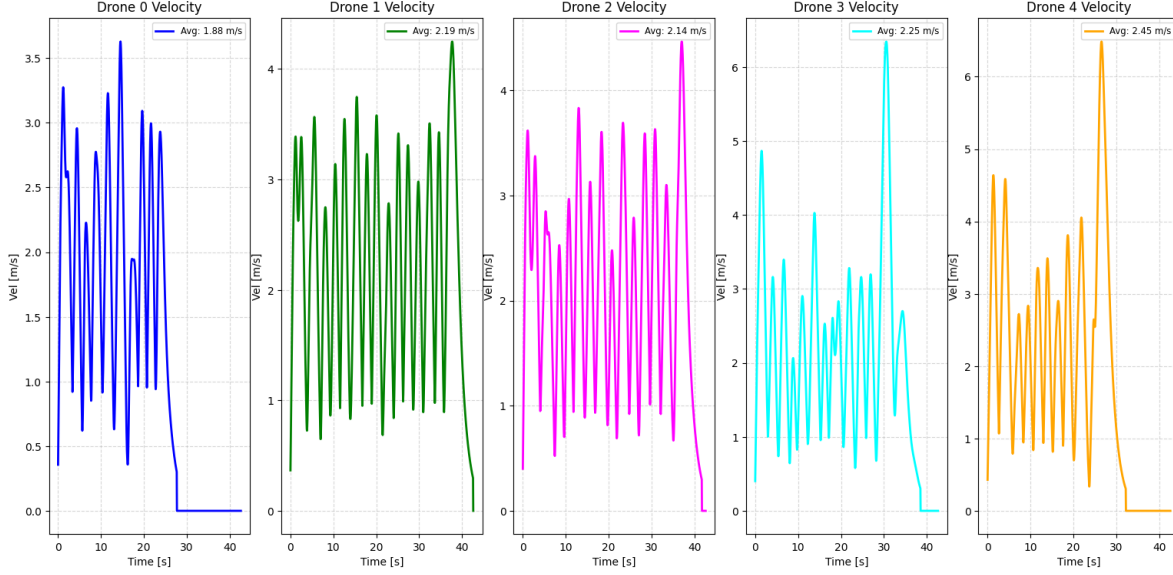


Figure 5: Velocities of the drones.

For performance evaluation, average velocities and accelerations were calculated strictly during the active mission phase, explicitly excluding the terminal return-to-base maneuvers which was not optimized in our implementation. Isolating the active flight time establishes a more accurate and convenient baseline for comparison against the results obtained with the proposed Early Switching improvements.

To improve speed near the target points and overall smoothness an **Early Switching** technique was employed, as treated in the following section.

Table 1: Simulation scenario and MPC parameters.

Parameter	Value	Description
Workspace size	$40 \times 40 \times 20$ m	Dimensions of the simulated environment
Number of obstacles	40	Cylindrical obstacles randomly generated in the workspace
Obstacle radius	$[0.5, 1.5]$ m	Range of obstacle radii
Obstacle height	$[4, 18]$ m	Range of obstacle heights
Number of UAVs	5	Size of the swarm
Maximum velocity	5 m/s	UAV velocity bound
Maximum acceleration	2 m/s ²	UAV acceleration bound
Camera FOV	84°	Camera angular field of view
Overlap factor	0.3	Coverage redundancy factor
Safety distance	0.5 m	Minimum obstacle clearance
Prediction horizon	50	MPC horizon length
Sampling time	0.05 s	MPC discretization step

7 Improvements

To mitigate the stop-and-go actuation near the waypoints, increase cornering speeds, and thereby enhance the overall smoothness of the trajectory, an **Early Switching** logic was introduced. This approach prematurely validates a waypoint by expanding the acceptance threshold radius. As a result, the UAVs experience substantial improvements in kinematic smoothness and mission times, incurring only a marginal 0.15% reduction in average coverage performance. Alongside the switching logic, a secondary analysis was conducted to evaluate the impact of the overlap factor, which was introduced during the initial waypoint generation phase. Table 2 summarizes the offline trajectory analysis across 20 maps, comparing the Normal and Early Switching logics under overlap factors of 0.1, 0.3, and 0.5.

Table 2: Offline Trajectory Analysis: Normal vs Early Switching Logic Across 20 Maps and Overlap Factors = (0.1, 0.3, 0.5)

Metric	Overlap Factor	Method	Drone 0	Drone 1	Drone 2	Drone 3	Drone 4
Speed (m/s)	0.1	Normal	1.43	1.50	1.34	1.39	1.41
		Early	2.13	2.09	2.01	2.05	2.06
	0.3	Normal	1.31	1.35	1.32	1.36	1.41
		Early	1.94	2.01	1.95	1.97	2.05
	0.5	Normal	1.34	1.33	1.39	1.30	1.36
		Early	1.97	1.99	2.02	1.94	1.99
Energy (J)	0.1	Normal	3255	2910	3286	3140	3235
		Early	3238	3216	3241	3227	3241
	0.3	Normal	3622	3494	3837	3397	3931
		Early	3689	3673	3699	3669	3703
	0.5	Normal	4446	4283	4482	4239	4676
		Early	4428	4412	4423	4416	4434
Time (s)	0.1	Normal	31.9	28.4	32.1	30.7	31.6
		Early	27.6	24.4	27.6	26.4	27.2
	0.3	Normal	35.4	34.3	37.6	33.3	38.5
		Early	30.2	29.1	31.9	28.3	32.9
	0.5	Normal	43.7	42.2	44.1	41.7	46.0
		Early	36.8	35.5	36.9	35.0	38.5
Coverage (%)	0.1	Normal					96.62%
		Early					97.38%
	0.3	Normal					99.32%
		Early					99.16%
	0.5	Normal					99.88%
		Early					99.85%

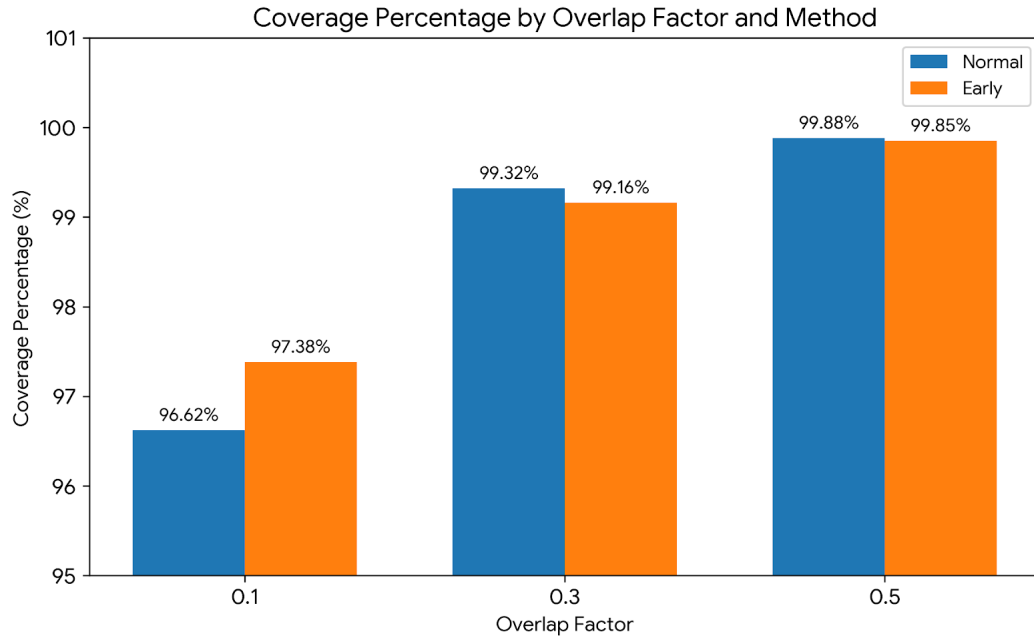


Figure 6: Comparison of the algorithm with and without early switching.

The kinematic improvements yielded by the Early Switching technique are visually depicted in Figure 7, which illustrates the velocity profiles mapped to the drones' trajectories.

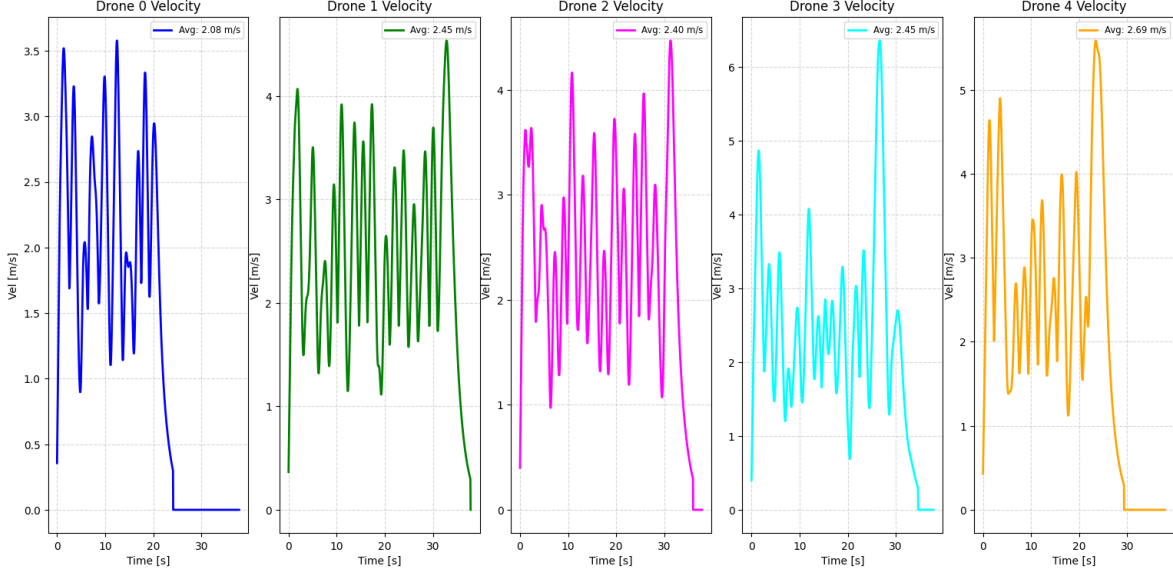


Figure 7: Velocities of the drones on the same map with the early switching technique.

Regarding the overlap parameter, the data indicates a clear trade-off. Increasing the overlap factor to 50% yields an average coverage gain of roughly 0.5%, but incurs a substantial penalty in energy consumption. Conversely, reducing the overlap to 10% results in an unacceptable degradation of overall coverage performance. Consequently, the standard overlap factor of 30% emerges as the optimal configuration, offering the best balance between area coverage and energy efficiency.

8 Robustness Analysis

A quick robustness analysis was performed, adding a gaussian noise of 20 centimeters on each component of the position vector for each drone, over 25 maps. Such error is enough to well represent a well implemented Sensor Fusion architecture in modern drones, equipped with IMU, camera and GNSS. The latter alone in such cluttered environments wouldn't be reliable enough. Also, we define a collision when the position of the drone is $\leq 1/4 * d_{safe}$ and as emergency braking maneuver the applied input (acceleration) set to $-current_velocity$ of the drone when the optimizer fails.

Table 3: Comparison of Normal vs. Early Switching Algorithms with 0.2 meters of noise on the position (Mean $\pm$ Std)

	Normal	Early
Speed (m/s)	1.272 ± 0.161	1.943 ± 0.204
Cumulative Squared Jerk (m^2/s^5)	7665.167 ± 6897.420	7349.713 ± 6395.022
Miss Distance (m)	0.230 ± 0.030	0.368 ± 0.054
Energy (J)	3893.129 ± 852.92	4228.548 ± 1181.123
Flight Time (s)	38.135 ± 8.427	33.565 ± 7.669
Collisions	0.000	0.000
Coverage (%)	99.417 ± 0.320	99.203 ± 0.397
Solve Time (s)	0.018 ± 0.004	0.020 ± 0.006
Success Rate (%)	99.200	99.200

Interestingly, coverage percentage increased with respect to the nominal case but the edges of the map or the areas inside multiple obstacles remain unseen, most likely due to the waypoint creation logic.

It is important to evidence that still 0 collisions occurred, and that the recursive feasibility was almost always recovered via the safety braking maneuver when the solver occasionally failed to find a solution.

The only failure case for the algorithm on the tested maps occurred, in some very specific cases, when a drone got stuck because it wasn't able to find any feasible trajectory. This occurred when it reached very cluttered regions of the map where the obstacles density didn't allow the drone to reach the next waypoint. This led the Cumulative Squared Jerk standard deviation to increase dramatically even if such failures were just outliers. Things slightly change when increasing the noise up to 30 centimeters:

Table 4: Comparison of Normal vs. Early Switching Algorithms with 0.3 m noise on each component of the position. (Mean $\pm$ Std)

	Normal	Early
Speed (m/s)	1.270 $\pm$ 0.167	1.932 $\pm$ 0.193
Jerk (m^2/s^5)	16348.078 $\pm$ 16652.207	14164.667 $\pm$ 9360.812
Miss Distance (m)	0.267 $\pm$ 0.386	0.372 $\pm$ 0.057
Energy (J)	4152.466 $\pm$ 1142.470	4490.363 $\pm$ 1147.865
Flight Time (s)	40.636 $\pm$ 11.269	35.135 $\pm$ 8.122
Collisions	0.000	0.000
Coverage (%)	99.078 $\pm$ 1.859	99.274 $\pm$ 0.379
Solve Time (s)	0.023 $\pm$ 0.005	0.045 $\pm$ 0.094
Success Rate (%)	98.400	99.200

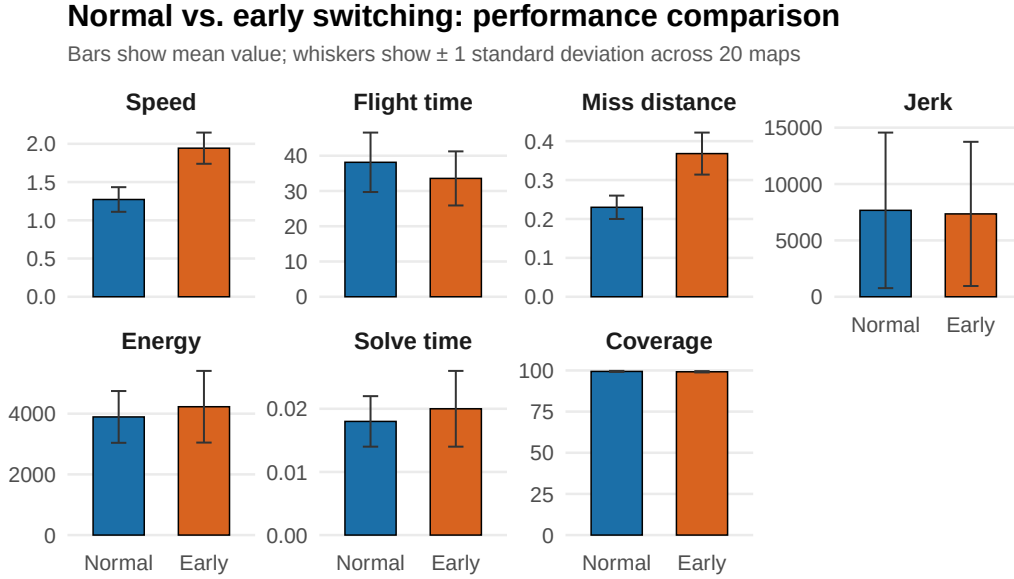


Figure 8: Normal vs Early Performance Comparison

The collected statistics prove that from this amount of noise upwards the algorithm gets stuck in possible exceptionally cluttered areas, leading to a substantial increase of energy consume and variations of flight time.

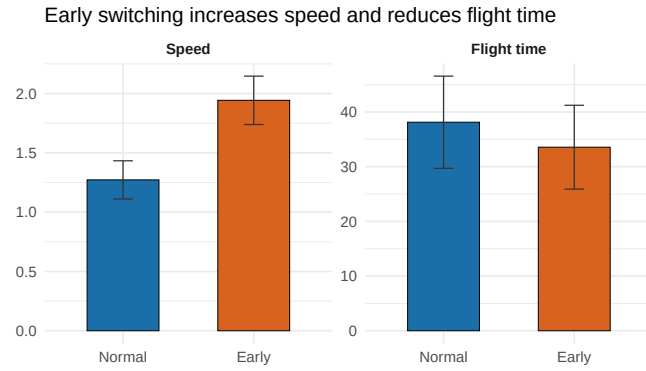


Figure 9: Normal vs Early switching: speed and flight time

Nevertheless, coverage performance did not degrade and the UAVs never collided. An example can be seen in the following image:

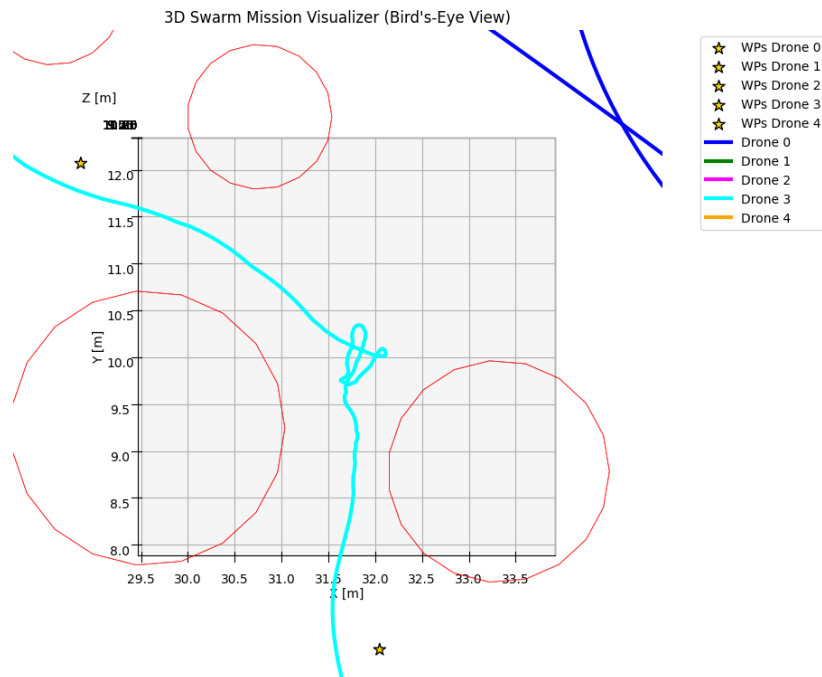


Figure 10: The drone gets stuck but after a couple of tentative path plannings finds a way out.

9 Conclusions

The simulation results demonstrate that the proposed DMPC architecture is highly robust for this class of multi-agent navigation problems. The way-point generation procedure serves as an effective baseline for coverage maximization, allowing the UAVs to achieve near-optimal mapping performance. Furthermore, the integration of the Early Switching logic significantly improves overall energy efficiency and trajectory smoothness without incurring any meaningful loss in area coverage.

A primary assumption of this current formulation is that the map and the dimensions of its obstacles are perfectly known *a priori*. Prior to real-world deployment, an empirical study must be conducted regarding the effect of spatial uncertainty on the obstacle safety radii. While modern SLAM pipelines can estimate these parameters dynamically, the resulting data is inherently subject to sensor noise and state estimation uncertainty, which must be rigorously accounted for in the safe distance constraints.

Future work will also focus on bridging the gap between this simplified kinematic model and actual flight hardware. Utilizing a high-fidelity physical simulation environment, the optimization problem could be expanded to incorporate full rigid-body dynamics (Newton-Euler formalism). This would better account for the complex aerodynamic effects of UAV actuation, provided that a final control allocation step is implemented to map the solver’s optimal accelerations to individual rotor commands. Consequently, a strict computational analysis would be required to ensure that this problem remains solvable in real-time on embedded flight controllers.

Finally, while this DMPC framework successfully navigates agents through complex, known environments, its reliance on a static map means it currently acts as a mid-level trajectory planner. For true autonomy in unstructured environments, this architecture must be coupled with a low-level reactive planner to seamlessly account for dynamic, unmapped obstacles and eventual environmental discrepancies between the map and the real world.